
Übungsblatt 6

Abgabe: 01.06.2026, 10:00 Uhr (im Digicampus via VIPS: .java-Dateien für Code, .uxf für UML, .pdf für alles andere)

- Dieses Übungsblatt muss im Team abgegeben werden (Einzelabgaben sind nicht erlaubt!).
- Die **Zeitangaben** geben zur Orientierung an, wie viel Zeit für eine Aufgabe später in der Klausur vorgesehen wäre; gehen Sie davon aus, dass Sie zum jetzigen Zeitpunkt wesentlich länger brauchen und die angegebene Zeit erst nach ausreichender Übung erreichen.

* leichte Aufgabe / ** mittelschwere Aufgabe / *** schwere Aufgabe

Aufgabe 21 (*Streams und Lambda-Ausdrücke, 25 Minuten*)

a) (**, 7 Minuten)

Schreiben Sie ein übersetzbares Java-Programm, das

- unter Benutzung der `doubles`-Methode der Klasse `Random` ein `DoubleStream`-Objekt mit 100 Zufallszahlen zwischen 0 (einschließlich) und 10 (ausschließlich) erzeugt,
- mit einem Prädikat alle Zahlen mit einem Wert kleiner als 5 entfernt und
- das Produkt der verbleibenden Zahlen mithilfe eines Lambda-Ausdrucks für die funktionale Schnittstelle `DoubleBinaryOperator` und der Methode `reduce` berechnet und ausgibt.

b) (**, 3 Minuten)

Passen Sie Ihre Lösung aus a) so an, dass anstelle von `reduce` eine Objektmethode der Klasse `DoubleStream` ohne Parameter verwendet wird, um die Summe der verbleibenden Zahlen zu bestimmen. Was passiert, wenn Sie die gesuchte Objektmethode der Klasse `DoubleStream` nach der Anwendung der `reduce`-Methode auf den `DoubleStream` anwenden? Warum ist es nicht möglich, beide Möglichkeiten in einer Programmklasse auf demselben Stream auszuführen? Beantworten Sie die Fragen als Kommentar in der zu dieser Teilaufgabe gehörenden Programmklasse.

c) (**, 7 Minuten)

Schreiben Sie ein übersetzbare Java-Programm, das unter Benutzung eines geeigneten Streams die **Gaußsche Summenformel** überprüft: Sie besagt, dass die Summe der ersten n natürlichen Zahlen immer folgenden Wert hat:

$$0 + 1 + \dots + n = \sum_{k=0}^n k = \frac{n \cdot (n + 1)}{2}$$

Erzeugen Sie dazu eine Zufallszahl zwischen 1 und 4000 (jeweils einschließlich) sowie einen geeigneten `IntStream` und geben Sie sowohl das Ergebnis der Summe der n ersten natürlichen Zahlen mithilfe des `IntStream` als auch den Wert des angegebenen Bruchs aus.

d) (**, 8 Minuten)

Schreiben Sie ein übersetzbare Java-Programm, das

- mithilfe eines `IntSuppliers` und einer entsprechenden Klassenmethode der Klasse `IntStream` ein `IntStream` Objekt mit unbegrenzt vielen ganzen Zufallszahlen erzeugt,
- dann den Stream auf 10 Zahlen begrenzt,
- den Stream in ein `Stream<Integer>`-Objekt verwandelt und
- mit einer geeigneten `Stream`-Methode und der Klassenmethode `Comparator.reverseOrder` die Zahlen in absteigender Sortierung jeweils in einer eigenen Zeile ausgibt.

Aufgabe 22 (Maps, 23 Minuten)

In dieser Aufgabe lernen Sie die Datenstruktur `Map` kennen. Maps sind parametrisierte Klassen und verwalten *Key-Value*-Paare: Sie ermöglichen es, zu beliebigen Objekt-Typen weitere Informationen in Form von Objekten zu speichern, mit zwei Einschränkungen:

- Kein *Key* kann doppelt vorkommen, genauer: Für jeden *Key* liefert `key.equals(otherKey)` `false` für alle anderen *Keys* `otherKey`.
- Für jeden *Key* ist höchstens ein *Value* hinterlegt.

Die zweite Einschränkung lässt sich durch die allgemein gehaltene Form der Schnittstelle leicht umgehen: Auch eine `ArrayList` ist ein Objekt und kann als *Value* eines Schlüssels verwendet werden.

Mit diesen Vorinformationen können Sie nun die folgenden Teilaufgaben bearbeiten. Die wichtigsten Methoden der Schnittstelle `Map<K, V>` sind:

- `Set<K> keySet()`: Gibt ein `Set`-Objekt zurück, das alle in der Map enthaltenen *Keys* enthält.
- `V put(K key, V value)`: Setzt den Wert `value` für den *Key* `key`.
- `V get(Object key)`: Gibt den Wert zurück, der für den *Key* `key` hinterlegt ist, oder `null`, wenn für `key` nichts hinterlegt ist
- `Collection<V> values()`: Gibt ein `Collection`-Objekt zurück, das alle von der Map verwalteten *Values* enthält.

a) (Maps kennenlernen, **, 8 Minuten)

Erstellen Sie ein streng typisiertes `HashMap`-Objekt für *Character-Integer*-Paare und eine Variable `map`, welche dieses Objekt referenziert. Iterieren Sie über die Kommandozeilenparameter des Programms und berechnen Sie, wie oft jedes enthaltene Zeichen vorkommt und speichern Sie die Anzahlen in der Map. Geben Sie für alle enthaltenen Zeichen die jeweilige Anzahl der Vorkommen aus. Achten Sie beim Befüllen der Map darauf, dass die `get`-Methode für *Keys*, die nicht vorhanden sind, `null` zurückgibt.

b) (*Gesetz der großen Zahlen*, ***, 15 Minuten)

In dieser Teilaufgabe lernen Sie das „Gesetz der großen Zahlen“¹ praktisch kennen und beantworten dabei folgende Frage: Wenn die `nextInt(int bound)`-Methode eines `Random`-Objekts gleichverteilt Werte aus dem Intervall $[0, bound[$ liefert, wie viele Werte müssen generiert werden, bis das Ergebnis tatsächlich (näherungsweise) gleichverteilt ist? Schreiben Sie dazu eine Klassenmethode `averageRelativeDeltaFromMean`, die ein `Map<Integer, Integer>`-Objekt erwartet. Die *Keys* der Map sind die zufällig generierten Werte aus dem Intervall $[0, bound[$ und die *Values* sind die jeweilige Anzahl, wie oft der Wert vorgekommen ist. Die Methode soll

- zuerst den **Durchschnittswert** *avg* der *Values* für alle *Keys* berechnen,
- für jeden *Value* *x* die **relative absolute** Abweichung $|avg - x|$ von diesem Durchschnittswert berechnen und dann
- den Durchschnitt dieser relativen absoluten Abweichungen für alle *Values* berechnen und zurückgeben.
- Für einen *Value* *x* und einen Durchschnittswert *avg* berechnet sich die relative absolute Abweichung als $\frac{|avg-x|}{avg}$.

Beispiel: Für *bound* = 3 und eine Map {0->3, 1->8, 2->4} mit den *Key-Value*-Paaren *key*->*value* soll die Funktion also folgenden Werte berechnen:

$$avg = \frac{3 + 8 + 4}{3}$$
$$averageRelativeDeltaFromMean = \frac{|avg - 3| + |avg - 8| + |avg - 4|}{3}$$

Für die `main()`-Methode

- erzeugen Sie ein `HashMap<Integer, Integer>`-Objekt und initialisieren Sie die Werte für die *Keys* 0 bis 9 mit 0.
- Berechnen Sie dann 10 zufällige Zahlen zwischen 0 und 9 (jeweils einschließlich) und zählen Sie deren Vorkommen in Ihrer Map.
- Berechnen Sie die durchschnittliche relative Abweichung und geben diese aus.
- Erzeugen Sie dann so lange zufällige Zahlen zwischen 0 und 9 (jeweils einschließlich), bis die durchschnittliche relative Abweichung weniger als 0.01 beträgt.
- Geben Sie dann die Anzahl der berechneten Zahlen aus, indem Sie mit der `values()`-Methode des `HashMap`-Objekts über die enthaltenen Werte iterieren.

Führen Sie das Programm mehrmals aus und betrachten Sie die ausgegebenen Werte.

¹https://de.statista.com/statistik/lexikon/definition/58/gesetz_der_gro_c3_9fen_zahl/

Aufgabe 23 (*Exceptions abfangen und werfen, 28 Minuten*)

a) (**, 10 Minuten)

Laden Sie die im Digicampus gegebene Klasse `FaultyCode` herunter. Sorgen Sie durch maximal spezifisches Abfangen aller auftretenden Exceptions dafür, dass das Programm fehlerfrei terminiert. Führen Sie dazu das Programm aus und verwenden Sie die ausgegebenen Stack Traces, um nach und nach alle Anweisungen, die Exceptions werfen, mit einem try-catch Block umgeben und mit geeigneten Methoden die Klasse und die enthaltene Fehlermeldung aller geworfenen Ausnahmen auf der Kommandozeile ausgeben. Umgeben Sie jeden Aufruf, der eine Exception wirft, mit einem eigenen try-catch Block. Lassen Sie die Anweisungen des Programms sonst unverändert!

b) (***, 18 Minuten)

Schreiben Sie eine Klasse `UnitConverter`, die einen primitiven Einheitenkonverter implementiert. Der Konverter soll in der Lage sein, einen nicht-negativen reellwertigen Wert von einer Quelleinheit in eine Zieleinheit umzurechnen. Unterstützt werden dabei die Längeneinheiten `"mm"`, `"cm"`, `"m"`, `"km"`, `"in"`, `"ft"`, `"mi"` sowie die Masseneinheiten `"mg"`, `"g"`, `"kg"`, `"oz"`, `"lb"`. Quell- und Zieleinheit müssen derselben Kategorie angehören. In Digicampus finden Sie eine Vorlage für die Klasse, in der die Werte zur Umrechnung bereits in zwei `HashMap`-Objekten hinterlegt sind. Die Ein- und Ausgabe soll über die Kommandozeile erfolgen. Pro Zeile soll eine Umrechnung `"<Wert> <Quelleinheit> <Zieleinheit>"` eingegeben werden, also z.B. `"100 km mi"`. Das Programm soll

- zu Beginn kurz über seine Benutzung informieren,
- solange zeilenweise Umrechnungen einlesen, bis die Zeichenkette `"EXIT"` eingegeben wird,
- die Eingabe in den Wert, die Quelleinheit und die Zieleinheit aufspalten und die Umrechnung mit ihrem Ergebnis ausgeben.

Beachten Sie außerdem folgende Vorgaben zur Implementierung:

- Implementieren Sie für die Berechnung des Ergebnisses eine eigene Klassenmethode, die auch das Ausgeben des Ergebnisses übernimmt.
- Verwenden Sie für das Aufteilen der Eingabe den regulären Ausdruck `"\\s+"` zusammen mit einer geeigneten Objektmethode der Klasse `String`.
- Bei der Nutzereingabe können verschiedene Fehler auftreten. Behandeln Sie die folgenden Fehler jeweils mit dem Werfen der angegebenen Ausnahme:
 - `IllegalArgumentException`: Es werden nicht genau drei Bestandteile übergeben
 - `IllegalArgumentException`: Quell- und Zieleinheit sind nicht beide bekannt oder gehören unterschiedlichen Kategorien an
 - `NumberFormatException`: Es wird kein gültiger reellwertiger Wert als Operand übergeben
 - `NumberFormatException`: Es wird ein negativer Wert übergeben

Wenn möglich, reichen Sie die von API-Methoden geworfenen Ausnahmen weiter.

- Behandeln Sie die auftretenden Ausnahmen in der `main`-Methode maximal spezifisch und geben Sie jeweils eine passende Fehlermeldung auf dem Standard-Fehlerstrom aus.

Aufgabe 24 (*Eigene Ausnahmeklasse, 20 Minuten*)

In dieser Aufgabe sollen Sie eine Datenklasse **Temperature** für Temperaturen implementieren, die durch eine eigene Ausnahmeklasse vor ungültigen Werten geschützt wird. Eine Temperatur hat einen reellwertigen Wert in Grad Celsius. Da der absolute Nullpunkt bei $-273,15\text{ }^{\circ}\text{C}$ liegt, sollen kleinere Werte nicht zugelassen werden.

a) (**, Ausnahmeklasse implementieren, 6 Minuten*)

Implementieren Sie eine geprüfte Ausnahmeklasse **TemperatureException**. Sie soll zusätzlich zu einer Fehlermeldung den ungültigen Temperaturwert speichern und über eine Getter-Methode zur Verfügung stellen. Überschreiben Sie außerdem die **toString**-Methode so, dass sie die Standardausgabe der Oberklasse um den ungültigen Wert ergänzt.

b) (***, Datenklasse implementieren, 8 Minuten*)

Implementieren Sie die Datenklasse **Temperature** mit einem reellwertigen Attribut für den Temperaturwert in Grad Celsius. Temperaturwerte dürfen nicht kleiner als der absolute Nullpunkt von $-273.15\text{ }^{\circ}\text{C}$ sein. Die Klasse soll

- passende Checker-, Getter- und Setter-Methoden für den Temperaturwert anbieten, wobei der Setter bei einem ungültigen Wert eine **TemperatureException** mit einer aussagekräftigen Fehlermeldung und dem ungültigen Wert wirft,
- einen Konstruktor besitzen, der den initialen Temperaturwert übernimmt und eine auftretende Ausnahme weiterreicht,
- die **toString**-Methode passend überschreiben.

Verwenden Sie den Setter auch im Konstruktor, damit der initiale Wert ebenfalls geprüft wird.

c) (**, Programmklasse implementieren, 6 Minuten*)

Implementieren Sie eine Programmklasse **WeatherStation**, mit der Sie Ihre Klassen testen können:

- Erzeugen Sie eine **ArrayList** von 20 **Temperature**-Objekten mit $20\text{ }^{\circ}\text{C}$ als Anfangswert.
- Versuchen Sie für jedes Objekt der Liste, mit Hilfe eines **Random**-Objekts einen zufälligen reellwertigen Wert aus dem Intervall $[-300, 50)$ zu setzen. Fangen Sie dabei auftretende **TemperatureExceptions** ab und geben Sie eine passende Fehlermeldung mit dem ungültigen Wert auf dem Standard-Fehlerstrom aus, ohne das Programm abzubrechen.
- Geben Sie am Ende alle Temperaturen der Liste auf der Kommandozeile aus.